TEST PLANNING AND AUTOMATION

Testing can be considered as a project on its own, it has to be planned, executed, tracked, and periodically reported on.

TESTING PLANNING

It is rightly sad that " Failing to plan is planning to fail ".Therefore testing also requires proper planning for appropriate execution.Testing planning includes:

1. Preparing a Test Plan
2. Scope Management: deciding Features to be Tested/Not tested
3. Deciding Test Approach/Strategy
4. Setting up Criteria for Testing
5. Identifying Responsibilities, Staffing, and Training Needs
6. Identifying Resources requirements
7. Identifying Test Deliverables
8. Testing Tasks : Size and Effort Estimation
9. Activity Breakdown and Scheduling
10. Communication Management
11. Risk Management

1. Preparing a Test Plan

Testing – like any project – should be driven by a plan. The test plan acts as the anchor for the execution, tracking, and reporting of the entire testing project and covers

- What needs to be tested
- How the testing is going to be performed
- What resources are needed for testing
- The time lines by which the testing activities will be performed.
- Risks that may be faced in all of the above.

2. Scope Management : Deciding Features to be Tested/Not Tested

Scope management pertains to specifying the scope of a project. For testing, scope management entails

- Understanding what constitutes a release of a project;
- Breaking down the release into features;
- Prioritizing the features for testing;
- Deciding which features will be tested and which will not be; and

- Gathering details to prepare for estimation of resources for testing.

It is always good to start from the end – goal or product – release perspective and get a holistic picture of the entire product to decide the scope and priority of testing.

The following factors drive the choice and prioritization of features to be tested.

- Features those are new and critical for the release.
- Features whose failures can be catastrophic.
- Features that are expected to be complex to test.
- Features which are extensions of earlier features that have been defect prone.

3. Deciding test approach/strategy

This include identifying

- What type of testing would you use for testing functionality?
- What are the configurations or scenarios for testing the feature?
- What integration testing would you do to ensure these features work together?
- What localization validations would be needed?
- What "non-functional" tests would you need to do?

4. Setting up Criteria for Testing

There must be clear entry and exit criteria for different phases of testing. There may also be entry criteria for the entries testing activity to start. The completion/exit criteria specify when a test cycle or a testing activity can be deemed complete.

5. Identifying Responsibilities, Staffing, and Training Needs

Scope management identifies what needs to be tested. The test strategy outlines how to do it. The next aspect of planning is the who part of it. Identifying responsibilities, staffing, and training needs addresses the aspect.

A testing project require different people to play different roles. As discussed in the previous two chapters, there are the roles of test engineers, test leads, and test managers. There is also role definition on the dimensions of the modules being tested or the types of testing. These different roles should complement each other. The different role definitions should

- Ensure there is clear accountability for a given task, so that each person knows what he or she has to do;
- Clearly list the responsibilities for various functions to various people, so that everyone knows how his or her work fits into the entire project;
- Complement each other, ensuring no one steps on an others toes; and
- Supplement each other, so that no task is left unassigned.

Role definitions should not only address technical roles, but also list the management and reporting responsibilities.

Staffing is done based on estimation of effort involved and the availability of time for release.

It may not always be possible to find the perfect fit between the requirements and the skills available. In case there are gaps between the requirements and availability of skills, they should be addressed with appropriate training programs.

6. Identifying Resource Requirements

As a part of planning for a testing project, the project manager (or test manager) should provided estimates for the various hardware and software resources required. Some of the following factors need to be considered.

- Machine configuration (RAM, processor, disk, and so on) needed to run the product under test.
- Overheads required by the test automation tool, if any.
- Supporting tools such as compilers, test data generators, configuration management tools, and so on.
- The different configurations of the supporting software (for example, OS) that must be present
- Special requirements for running machine-intensive tests such as load tests and performance tests
- Appropriate number of licenses of all the software.

In addition to all of the above, there are also other implied environmental requirements that need to be satisfied. These include office space, support functions (like HR), and so on.

7. Identifying Test Deliverables

The test plan also identifies the deliverables that should come out of the test cycle/testing activity. The deliverables includes the following, all reviewed and approved by the appropriate people.

- The test plan itself (master test plan, and various other test plans for the project)
- Test case design specifications
- Test cases, including any automation that is specified in the plan.
- Test logos produced by running the tests
- Test summary reports.

8. Testing Tasks: Size and Effort Estimation

The scope identified above gives a broad overview of what needs to be tested. This understanding is quantified in the estimation step. Estimations happens broadly in three phases.

- Size estimation
- Effort estimation
- Schedule estimation.

Size estimation: It quantifies the actual amount of testing that needs to be done several factors contribute to the size estimate of testing project.

- Size of the product under test. This obviously determines the amount of testing that needs to be done. The larger the product, in general, grater is the size of testing to be done. Some of the measurement of the size of product under test are as follows;
  1. Line of Code (LOC).
  2. A function point (FP).
  3. No of screens reports or transactions.
- Extent of automation required when automation is involved, the size of work to be done for testing increases.
- No of platforms and inter-operability environments to be tested. If a particular product is to be tested under several different platforms or under several different configurations, then the size of the testing task increases.

Effort Estimation: Estimating effort is important because often effort has a more direct influence on cost than size. The other factors that drive the effort estimate are as follows:

- Productivity data. Productivity refers to the speed at which for various activities of testing can b e carried out. This is based on historical data available in the organisation. Productivity data can be further classified into the number of test cases that can be developed per day (on some unit

time), the number of test cases that can be run per day, the number of pages of documentation that can be tested per day, and so on.

- Reuse opportunities : If the test architecture has been designed keeping reuse in mind, then the effort required to cover a given size of testing can come down.
- Robustness of process: Reuse is a specific example of process maturity of an organisation. Existence of well-defined process will go a long way in reducing the effort involved in any activity. For example;
  1. Well-documented standards for writing test specifications, test scripts etc.
  2. Proven process for performing functions such as reviews and audits;
  3. Consistent ways of training people, and
  4. Objective ways of measuring the effectiveness of compliance to process

9. Activity Breakdown and Scheduling

Activity breakdown and schedule estimation entail translating the efforts required into specific time frames. The following steps make up this translation.

- Identifying external and internal dependencies among the activities.
- Sequencing the activities, based on the expected duration as well as on the dependencies
- Identifying the time required for each of the WBS activities, taking into account the above two factors
- Monitoring the progress in terms of time and effort
- Rebalancing schedules and resources as necessary.

External dependencies of an activity

- Availability of the product from developers;
- Hiring
- Training
- Acquisition of hardware/software required for training; and
- Availability of translated messages files for testing.

Internal dependencies

- Completing the test specification
- Coding/scripting the tests
- Executing the tests.

Scheduling: Based on the dependencies and the parallelism possible, the test activities are schedules in a sequences that helps accomplish the activities in the minimum possible time.

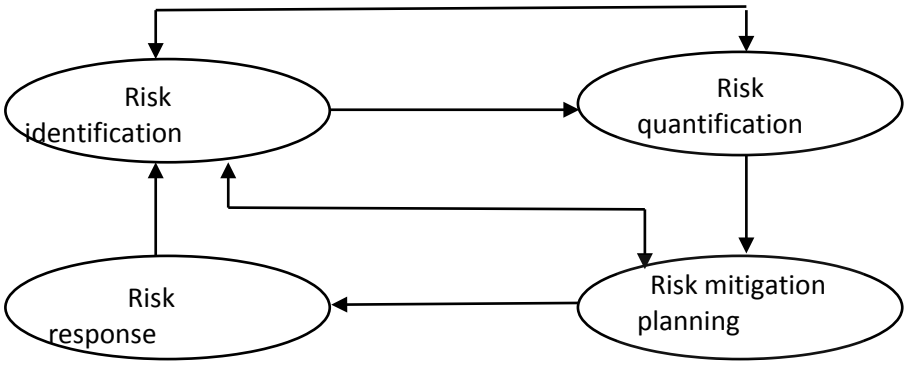
10. Communications Management

Communications management consists of evolving and following procedures for communication that ensure that everyone is kept in sync with the right level of detail.

11. Risk Management

Risks are uncertain events that could potentially affect a project's outcome. Risk management entails

- Identifying the possible risks
- Quantifying the risks;
- Planning how to mitigate the risks; and
- Responding to risks when they become a reality.



I) ***Risk identification*** consists of identifying the possible risks that may it a project. The following are some of the common ways to identify risks in testing.
   - Use of checklists
   - Use of organizational history and metrics
   - Informal networking across the industry.

II) ***Risk quantification*** deals with expressing the risk in numerical terms. There are two components to the quantification of risk. One is the probability of the risk happening and the other is the impact of the risk, if the risk happens.

III) ***Risk mitigation planning*** deals with identifying alternative strategies to combat a risk event, should that risk materialize.

IV) ***Risk response*** when the above three steps are carried out systematically and in a timely manner, the organization would be in a better position to respond to the risk, should the risks become a reality.

The following are some of the common risks encountered in testing projects and their characteristics.

- Unclear requirements
- Schedule dependence
- Insufficient time for testing
- "Show stopper" defects.
- Availability of skilled and motivated people for testing.
- Inability to get a test automation tool

TEST AUTOMATION

Automation Testing

What is Automation Testing?

Automation Testing is a technique that uses tools to write scripts and execute test cases. It is the best way to enhance the execution speed, effectiveness, and test coverage in software testing. Besides, it is cost-effective and helps find possible bugs quickly.

What are the types of Automation Testing?

Automation testing types

There are certain tests that should be automated using test automation tools. Some of them are as follows:

Unit Testing:

Unit testing is the process of testing the individual units of software, i.e., a function, module, method, or class, in isolation. It helps catch the bugs early in development and reduces the cost. It also enables code reusability and helps you quickly migrate your code and test into a new project.

Integration Testing:

Integration testing checks defects in the interaction between multiple modules when they are integrated. This can be done through API testing or the UI layer of the application.

Regression testing:

It is a continuous process to ensure that newly added functionality is working correctly without affecting other functionality in the application. It helps to ensure that a newly added feature will not introduce any significant bug issues which will break the application.

Performance testing:

Performance testing is about testing the speed, load time, stability, and scalability of an application. It ensures that an application performs as per the requirement.

Security testing:

Security testing is about the accessibility of an application and its user data. Various other tests are performed within security testing, for example, penetration testing, security scanning, etc.

The advantages of automation testing are given as follows **-**

Saves time

Automating the testing process helps the testing team to use less time to validate newly created features. For instance, in manual testing, there is a need to write thousand test cases for a calculator application, but automation makes the process much faster.

Productivity improvement

As during execution, automation tests do not require human intervention, so testing an application can be done late at night, and we can get the results next morning. Software developers and testers require less time on automation testing.

Accuracy improvement

In manual testing, there is a chance of mistakes whether you are an experienced testing engineer. The chances of errors may increase when testing a complex use case. But Automation testing reduces the chances of errors. There is good accuracy, as we will get the same result each time on performing the same test cases.

Test suite reusability

We can reuse the test scripts in automation testing, and we don't need to write the new test scripts again and again. These test cases can be used in various ways, as they are reusable. Reusability helps to reduce the cost and also eliminate the chances of human error.

Ability to test on various platforms

Automation testing allows the user to test the application on different web browsers and operating systems.

Running tests 24/7

In automation testing, we can start the testing process from anywhere in the world and anytime we want. It can also be done remotely if we don't have many approaches or the option to purchase them.

Early bug detection

By automation testing, it is easy to detect critical bugs in the initial phases of software development. It reduces the cost and helps us to spend fewer working hours to fix such problems. It increases the efficiency of the team.

Less human resources

Automation testing requires fewer people to perform a tedious manual test. To implement the automation test script, we need a test automation engineer who can write the test scripts to automate our tests.

Reduce the expenses

Automation testing is less expensive, as once the test scripts have been built, we can reuse them at any time without any extra cost. While manual testing is more expensive than automation, with manual monitoring, it is typical to execute experiments repeatedly.

Scalability of test cases

In manual testing, we require the involvement of the number of people and number of hours to scale up a project. Whereas the scalability of automation testing is higher, we need adding of test executors to the testing framework.

Consistency

Compared to manual testing, automation testing is more consistent and way faster than executing the regular monotonous tests that cannot be missed but may cause faults when tested manually.

Fast development and delivery

Automated tests can be executed repeatedly and completed rapidly. We do not have to wait for weeks to execute the tests; few hours are enough for execution. Switching from manual to automation reduces the waiting time and boosts development.

Easily execution of lengthy and complicated test cases

Execution of bug-prone and complex test cases is easier with automation testing. Test cases with reproducible steps lead to distraction and wrong assurances on testing them manually.

Some of the other benefits of automation testing are listed as follows -

- In comparison to manual testing, automation testing requires fewer resources.
- It makes load and performance testing, stress testing, and reliability testing possible.
- It is more reliable, as it reduces the occurrence of errors. It is reliable because it tests the application with the help of tools and test scripts.
- With automation testing, test engineers are free to focus on other work.
- It improves the testing coverage as the automatic execution of test cases is faster than manual execution.
- Automation testing allows the execution of test cases in a 24x7 environment.
- It enhances the knowledge of test engineers by producing a repository of different test cases.
- Batch execution is possible using automation testing because all the written scripts can be executed simultaneously.
- Automation testing is 70% faster than manual testing.

What Test Cases to Automate

It is impractical to automate all testing, so it is important to determine what test cases should be automated first.

You can get the most benefit out of your automated testing efforts by automating:

- Repetitive tests that run for multiple builds.
- Tests that tend to cause human error.
- Tests that require multiple data sets.
- Frequently used functionality that introduces high risk conditions.
- Tests that are impossible to perform manually.
- Tests that run on several different hardware or software platforms and configurations.

- Tests that take a lot of effort and time when manual testing.

Which test cases should you not automate

Subjective Validation

Subjective validation refers to the act of protecting the validity of words, statements, and initials. It's best performed manually because humans can quickly detect and provide critical feedback instead of automation, which will take time to write and run.

New Functionalities

Applications and software that are still under development will often require many changes in the underlying code. In such a scenario, every time the code is changed, the automated test script will need to be altered too. This alone makes test automation time-consuming and tedious.

Business-critical Functionalities

When building software, certain functions form the core. Automated testing shouldn't be used to test these functions. Test cases also require subject matter expertise that cannot be performed by automation software. Only qualified personnel should test these business-critical functionalities.

User Experience

UI tests are generally hard to automate. The scripts are complicated and require several changes to run successfully. Moreover, the human eye is best suited to test UI features such as resolution, formatting, etc.

Complex Functionalities

Some features of the application under test will have complex pathways. For instance, tests that require data modification and manipulation downstream are best suited for manual testing. No automation script can do justice to these test cases.

Quality Control

The final quality checks run on the software or application should also be left for manual testers. Automated tests are best suited to test for results that are predictable and fixed. Additionally, no automation solution can give the kind of feedback that a human tester can.

Test automation has proven to be a highly efficient method to shorten the SDLC but only when done right. Rushing in to automate every test will backfire and rob the

process of its efficiency. By ensuring that only the tests with the highest ROI are automated, organizations can get better results

Manual testing vs. Automation testing

Manual Testing	Automation Testing
Test cases are executed manually.	Test cases are executed with the help of tools.
Reliability is less.	Reliability is more.
It is less costlier.	It is more costlier.
For some test cases it consumes time.	As it is a machine it take less time to execute cases.
Human can make mistakes and hence accuracy is less.	Machine hardly makes mistakes(If it has been asked to do so).
As it include human intervention, it is beneficial to check ease for accessing the application.	It includes tools so unable to check usability or accessibility.
Using manual testing, it could be difficult to test the application on different Operating systems.	With the help of Automation testing, we can easily test the application on different Operating systems.
Sometimes it becomes difficult to execute all the test cases and it impacts test coverage.	In Automation testing we can achieve the test coverage target.
For Manual, it may find difficult test the application on different browsers.	Automation gives you advantage to test the software on different browsers. Selenium grid allow us to test the application on different browsers.

Design and Architecture for Automation

How does Automation Testing Work?

Organizations will implement Test Automation with a framework that will have standards, common practices, and testing tools. A good Automated Testing Framework includes standards for coding, methods for handling test data, object repositories, procedures for storing test results, or details on utilizing external resources.

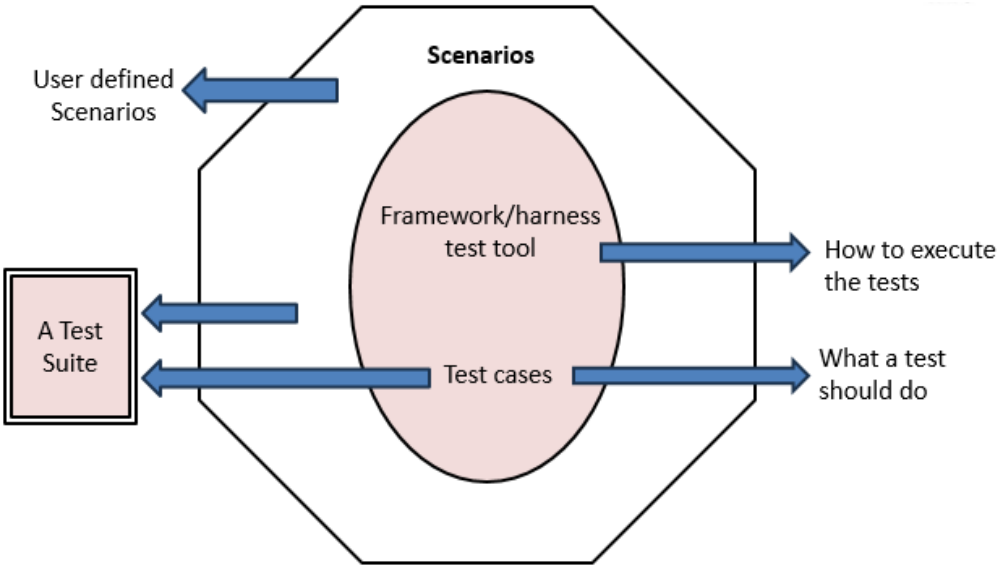


Figure: Framework for test automation

A framework is an abstraction in which software providing generic functionality can be selectively changed by additional user-written code, thus providing application-specific software . It isn't the easiest definition to digest, and one can usually struggle to actually tell a difference between framework and library. The key difference between framework and library, however, is who calls what. Long story short, when you use library, you call library's code. In comparison, when you use a framework, it calls your code.

Test automation frameworks by no means different. In general, a **test automation framework** is something that allows you to implement application-specific test automation solution by writing and injecting some additional code. Surprisingly, in test automation community by framework people often mean any test automation solution that happened to have any decent structure and design. Wikipedia as well supports this opinion

All in all, test frameworks designed to address following issues:

1.Test complexity

The simpler the test on the top level is, the easier it is to create, understand, communicate and maintain.

2. Test maintenance cost

Test automation isn't a thing that goes for granted, it has a price. One of the most painful prices for test automation is a maintenance cost. Well-designed framework should simplify maintenance.

3. Test execution time

Time is a money. Time to get a feedback from automated tests is also crucial – one wouldn't like to wait for several hours just in order to check that his commit wasn't wrong.

4. Reporting

Testing is about not finding bugs or accepting stories. Well, not exactly. What testing is really about – is providing an information. Test solution that has simple, maintainable tests, easy to maintain and enhance may be just thrown away if it produces shity reports that difficult to understand by non-technical stakeholders.

Scope of Automation

The first phase involved in product development is requirements gathering it is no different for test automation as the output of automation can also be considered as a product.

Some generic tips for identifying the scope for automation are:

- Identifying the types of Testing Amenable to Automation

- Stress, reliability, scalability and performance testing
- Regression tests
- Functional tests

- Automating areas less prone to change
- Automate tests that pertain to standards
- Management aspects in Automation

Top Automation Testing Tools

Many tools are available in the market, each with pros and cons. We have listed some of the best test automation tools below:

- **Testsigma**: Testsigma is an open-source and cloud-based test automation tool that makes it easy to write and execute tests using NLP (plain English). The best part – no coding knowledge is necessary.

- **Selenium**: Selenium is an open-source test automation tool for web apps. It's compatible with various programming languages, frameworks, browsers, and operating systems.

- **Appium**: Appium is an open-source test automation framework used to test mobile applications.

- **Watir**: Watir stands for web automation testing in ruby; it's a lightweight and open-source web automation testing tool.

- **Robotium**: Robotium is an open-source test automation tool that mainly tests the user interface of android applications.

Scope of Automation

The first phase involved in product development is requirements gathering it is no different for test automation as the output of automation can also be considered as a product.

Some generic tips for identifying the scope for automation are:

- Identifying the types of Testing Amenable to Automation

- Stress, reliability, scalability and performance testing
- Regression tests
- Functional tests

- Automating areas less prone to change
- Automate tests that pertain to standards
- Management aspects in Automation

Process of Test Automation

1. Convince the Management

No matter how much you are eager to discover and initiate test automation in your organization, you cannot do anything if your management is not convinced about the benefits test automation offers. It is a universal fact that test automation is expensive. The tools are expensive (HP QTP/UFT license cost around $8K per machine). There is a cost of a test automation architect or engineer (which, by the way, are expensive too). After that, the benefits of test automation cannot be seen immediately. You have to wait 2-3 months before your scripts are prepared, tested, and that can run reliably for you to test the application.

2. Finding Automation tool experts
There are two kinds of automation experts.

1. Automation architects
2. Automation engineers

Automation architects are a rare breed. They are hard to find, extremely expensive, and extremely necessary for the success of the automation project. These people are usually responsible to build automation frameworks.

3. Using the correct tool for automation

This point deserves its own article (and I will write one on that). This is another difficult step in the process of starting automation. There are various tools in the market, but you have to select those which are best for your application.

4. Training the Team
After tool selection and resource hiring, the next step is logically the training of the resources.

If manual testers are converted into automation engineers, they have to be trained on automation terminologies and concepts. If an automation architect is hired from outside, he must get knowledge about the product to test, the manual testing process, and what management is expecting.

5. Creating the test automation Framework
The biggest task for the automation architect is to come up with an automation framework that should support automated testing in the long run.

6. Developing an Execution Plan
The execution plan includes selecting which environments the scripts will be executed. The environment includes OS, Browser, and different hardware configurations.

For example, if the test case demands that it should check the website in 3 browsers, namely, Chrome, Firefox, and IE, then the automation team will write the script in such a manner that it will be able to execute in each browser.

7. Writing Scripts
When the framework is designed, the execution plan is known and resources are trained on the new tool, now it’s the right time to start writing scripts.

8. Reporting
The reporting feature is usually provided by the tool. But we can create custom reporting mechanisms like auto-emailing the results to management.

9. Maintenance of Scripts
If best programming practices are followed and the framework is good, then maintenance will not be a problem.

Maintenance usually occurs when there is a change request in an application. The scripts should immediately be updated to cope with that change to ensure flawless execution.

For example, if you are writing some text in the textbox through the script and now this text box becomes the drop-down list, we should immediately update the script.